Database Management Systems

Procedures

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Stored Procedures

- A procedure (often called a stored procedure) is a collection of pre-compiled SQL statements stored inside the database.
- It is a subroutine or a subprogram in the regular computing language.
- A procedure always contains a name, parameter lists, and SQL statements. We can invoke the procedures by using triggers, other procedures and applications such as Java , Python , PHP , etc.
- The following syntax is used for creating a stored procedure in MySQL.

```
DELIMITER &&
CREATE PROCEDURE procedure_name [[IN | OUT | INOUT] parameter_name datatype
    [, parameter  datatype]) ]
    BEGIN
        Declaration_section
        Executable_section
    END &&
DELIMITER ;
```

Stored Procedures

Stored procedures are useful in the following circumstances:

- If a database program is needed by several applications, it can be stored at the server and invoked by any of the application programs.
- This reduces duplication of effort and improves software modularity.
- Executing a program at the server can reduce data transfer and communication cost between the client and server in certain situations.
- These procedures can enhance the modeling power provided by views by allowing more complex types of derived data to be made available to the database users via the stored procedures.
- Additionally, they can be used to check for complex constraints that are beyond the specification power of assertions and triggers.

Stored Procedures

Modes of Procedure Parameters:

- IN: It is the default mode. It takes a parameter as input, such as an attribute. When we define it, the calling program has to pass an argument to the stored procedure. This parameter's value is always protected.
- OUT: It is used to pass a parameter as output. Its value can be changed inside the stored procedure, and the changed (new) value is passed back to the calling program. It is noted that a procedure cannot access the OUT parameter's initial value when it starts.
- INOUT: It is a combination of IN and OUT parameters. It means the calling program can pass the argument, and the procedure can modify the INOUT parameter, and then passes the new value back to the calling program.
- How to call a stored procedure?
- Syntax: CALL procedure_name (parameter(s))

PES UNIVERSITY

Consider this table

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975
4	104	Barack	maths	90	8769875325
5	105	Rinky	maths	85	6753157975
6	106	Adam	Science	92	7964225686
7	107	Andrew	Science	83	5674243757
8	108	Brayan	Science	85	7523416567
10	110	Alexandar	Biology	67	2347346438

PES UNIVERSITY

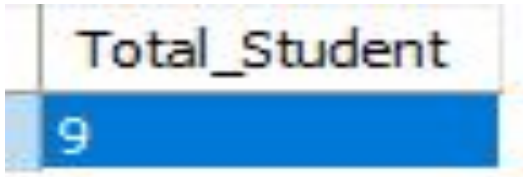
Example:

Procedure without ParameterSuppose we want to display all records of this table whose marks are greater than 70 and count all the table rows.

The following code creates a procedure named get_merit_students:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE`get_merit_student`()
BEGIN
    SELECT * FROM student WHERE marks > 70;
    SELECT COUNT(student_code) AS Total_Student FROM student;
END$$
DELIMITER ;
```

Output:



Total_Student
9

Stored Procedures

PES UNIVERSITY

Procedure without Parameter:

Example: Suppose we want to display all records of this table whose marks are greater than 70 and count all the table rows.

The following code creates a procedure named get_merit_students:

```
DELIMITER $$
CREATE PROCEDURE`get_merit_student`()
BEGIN
    SELECT * FROM student WHERE marks > 70;
    SELECT COUNT(student_code) AS Total_Student FROM student;
END$$
DELIMITER ;
```

Query: CALL get_merit_student();

Output:

Total_Student
9

Procedures with IN Parameter

In this procedure, we have used the IN parameter as **'var1'** of integer type to accept a number from users. Its body part fetches the records from the table using a **SELECT** statement and returns only those rows that will be supplied by the user. It also returns the total number of rows of the specified table. See the procedure code:

```
DELIMITER &&
CREATE PROCEDURE get_student (IN var1 INT)
BEGIN
    SELECT * FROM student_info LIMIT var1;
    SELECT COUNT(stud_code) AS Total_Student FROM student_info;
END &&
DELIMITER ;
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975

Stored Procedures

Query:

```
Call get_student(3);
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975

Query:

```
Call get_student(5);
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975
4	104	Barack	maths	90	8769875325
5	105	Rinky	maths	85	6753157975

Procedures with OUT Parameter

In this procedure, we have used the OUT parameter as the **'highestmark'** of integer type. Its body part fetches the maximum marks from the table using a **MAX() function**. See the procedure code:

```
DELIMITER &&
CREATE PROCEDURE `display_max_mark` (OUT highestmark INTEGER)
BEGIN
    SELECT max(marks) INTO highestmark FROM student;
END&&
DELIMITER ;
```

Query:

```
call display_max_mark(@output);
select @output;
```

Output:

@output
92

Procedures with INOUT Parameter:

In this procedure, we have used the INOUT parameter as **'var1'** of integer type. Its body part first fetches the marks from the table with the specified **id** and then stores it into the same variable var1. The var1 first acts as the IN parameter and then OUT parameter. Therefore, we can call it the INOUT parameter mode. See the procedure code:

```
DELIMITER &&
CREATE PROCEDURE `display_marks` (INOUT var1 INTEGER)
BEGIN
    SELECT marks INTO var1 FROM student WHERE stud_id = var1;
END&&
DELIMITER ;
```

Stored Procedures

Query:

```
SET @M = '3';
CALL display_marks(@M);
SELECT @M;
```

Output:

@M
70

Query:

```
SET @M = '4';
CALL display_marks(@M);
SELECT @M;
```

Output:

@M
90

How to delete/drop stored procedures in MySQL?

MySQL also allows a command to drop the procedure. When the procedure is dropped, it is removed from the database server also. The following statement is used to drop a stored procedure in MySQL:

DROP PROCEDURE [IF EXISTS] procedure_name;

Ex: DROP PROCEDURE display_marks;

We can verify it by listing the procedure in the specified database using the SHOW PROCEDURE STATUS command.

SHOW PROCEDURE STATUS [LIKE 'pattern' | WHERE search_condition]

Ex: SHOW PROCEDURE STATUS WHERE db = 'mystudentdb';

STORED PROCEDURE

```
CREATE PROCEDURE StudentMarksWithLoop()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE marks INT;
    DECLARE student_name VARCHAR(25);

    WHILE done = 0 DO
        SELECT marks, student_name INTO marks,
        student_name
        FROM student
        WHERE done = 0
        LIMIT 1;

        IF marks >= 50 THEN
            SELECT CONCAT(student_name, '
passed') AS Result;
        ELSE
            SELECT CONCAT(student_name, '
failed') AS Result;
        END IF;

        SET done = 1;
    END WHILE;
END //

DELIMITER ;
```

STORED PROCEDURE with While and if-else


```
CREATE PROCEDURE StudentMarksWithLoop()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE marks INT;
    DECLARE student_name VARCHAR(25);

    WHILE done = 0 DO
        SELECT marks, student_name INTO marks,
        student_name
        FROM student
        WHERE done = 0
        LIMIT 1;

        IF marks >= 50 THEN
            SELECT CONCAT(student_name, ' passed')
AS Result;
        ELSE
            SELECT CONCAT(student_name, ' failed')
AS Result;
        END IF;

        SET done = 1;
    END WHILE;
END //

DELIMITER ;
```

Note: This WHILE loop that continues to process students until the done flag is set to 1. Inside the loop, we retrieve the marks and student_name for one student at a time, check if they passed or failed, and then set the done flag to 1 to exit the loop after processing one student.

STORED PROCEDURE Vs FUNCTION

Stored Procedure	Function
Supports in, out and in-out parameters,i.e., input and output parameters	Supports only input parameters, no output parameters.
Stored procedures can call functions as needed	The function cannot call a stored procedure
There is no provision to call procedures from select/having and where statements	You can call functions from a select statement
Transactions can be used in stored procedures	No transactions are allowed
Can do exception handling by inserting try/catch blocks	No provision for explicit exception handling
Need not return any value	Must return a result or value to the caller
All the database operations like insert, update, delete can be performed	Only select is allowed

THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu